

Minimal 4-Query Evidence Set

Run these on **both** projects (source + restored) and paste results into your ticket.

<> SQL QUERY

<>  

```
-- Q1: Environment header
select
  current_database() as database_name,
  now() as checked_at_utc,
  current_setting('server_version') as pg_version;
```

<> SQL QUERY

<>  

```
-- Q2: Public tables (core signal)
select
  tablename
from pg_catalog.pg_tables
where schemaname = 'public'
order by tablename;
```

<> SQL QUERY

<>  

```
-- Q3: Critical row counts (edit table names if needed)
select 'app_user' as table_name, count(*)::bigint as exact_rows from public.app_user
union all
select 'employer' as table_name, count(*)::bigint as exact_rows from public.employer
union all
select 'caregiver' as table_name, count(*)::bigint as exact_rows from public.caregiver
order by table_name;
```

<> SQL QUERY

<>  

```
-- Q4: Public schema fingerprint (single-value compare)
select
  md5(
    string_agg(
      concat_ws(
        '|',
        c.table_schema,
        c.table_name,
        c.ordinal_position::text,
        c.column_name,
        c.data_type,
        coalesce(c.is_nullable, ''),
        coalesce(c.column_default, '')
      ),
      '' order by c.table_schema, c.table_name, c.ordinal_position
    )
  ) as public_schema_fingerprint
from information_schema.columns c
where c.table_schema = 'public';
```

What to conclude quickly

- If Q2 is empty on restored: restore failed for app data.
- If Q2 has tables but Q4 differs: schema mismatch/drift.
- If Q2 + Q4 match but Q3 is lower: likely older backup point.

If you share the two outputs, I can help you draft a precise support ticket text.

—    

The Assistant can make mistakes. Double check responses.

- Records created after backup/PITR target won't exist

Immediate fail conditions

If any of these happen, mark restore test as **FAILED**:

- `public` has zero tables
- Any required table is missing
- Fingerprint mismatch for critical tables (unless intentional migrations occurred after backup point)
- App smoke tests fail due to missing relations/data

Suggested pass criteria

Mark as **PASS** only if:

- All expected `public` tables exist
- Critical table schemas match
- Critical row counts are within expected time-window variance
- Basic app queries succeed (login lookup, primary dashboard query, etc.)

Incident packet for support (if failed)

Include:

- Source project ref
- Restored project ref
- Backup type (scheduled physical / PITR) and exact timestamp selected
- Output of:
 - query #2 (public tables)
 - query #3 (schema counts)
 - query #6 (public fingerprint)
 - query #5 (critical counts)
- Confirmation that restore completed "healthy" but `public` missing

Support URL: <https://supabase.com/dashboard/support/new>

Practical testing cadence

- Run this DR test monthly
- Keep one saved "golden" result from production baseline
- Re-run after major migrations or extension changes

—    

The Assistant can make mistakes. Double check responses.

Ask a follow up question...

Optional: one-shot JSON evidence bundle

This is handy for support tickets.

<> SQL QUERY

<>  

```
with public_tables as (  
  select tablename  
  from pg_catalog.pg_tables  
  where schemaname = 'public'  
) ,  
schema_counts as (  
  select  
    n.nspname as schema_name,  
    count(c.oid) filter (where c.relkind in ('r', 'p')) as table_count  
  from pg_namespace n  
  left join pg_class c  
    on c.relnamespace = n.oid  
  where n.nspname not like 'pg_%'  
    and n.nspname <> 'information_schema'  
  group by n.nspname  
) ,  
public_fp as (  
  select md5(  
    string_agg(  
      concat_ws(  
        '||',  
        c.table_schema,  
        c.table_name,  
        c.ordinal_position::text,  
        c.column_name,  
        c.data_type,  
        coalesce(c.is_nullable, ''),  
        coalesce(c.column_default, '')  
      ),  
      '||' order by c.table_schema, c.table_name, c.ordinal_position  
    )  
  ) as fp  
  from information_schema.columns c  
  where c.table_schema = 'public'  
)  
select jsonb_pretty(  
  jsonb_build_object(  
    'database', current_database(),  
    'checked_at_utc', now(),  
    'public_table_count', (select count(*) from public_tables),  
    'public_tables', (select jsonb_agg(tablename order by tablename) from public_tables),  
    'schema_table_counts', (  
      select jsonb_agg(  
        jsonb_build_object('schema_name', schema_name, 'table_count', table_count)  
        order by schema_name  
      )  
      from schema_counts  
    ),  
    'public_schema_fingerprint', (select fp from public_fp)  
  )  
) as recovery_evidence;
```

Ask a follow up question...

What I verified

I checked the restored project directly:

- `public` has 0 tables
- Only Supabase-managed schemas (`auth` , `storage` , `realtime` , etc.) have objects
- No active platform incidents were reported
- Branch-action logs were empty; Postgres log retrieval failed in this environment

So your observation is confirmed: **this restore does not contain your app tables.**

Why this is unexpected

Per Supabase docs, “Restore to a new project” from scheduled physical backups is supposed to include **database schema + data** (including `public`), not just `auth` .

Relevant docs:

- <https://supabase.com/docs/guides/platform/clone-project>
- <https://supabase.com/docs/guides/platform/backups>

Given that, this usually points to one of these:

- The restore was created from a backup that did not contain those tables (wrong source/timestamp)
- The source used for restore was not the intended production project
- A restore pipeline anomaly occurred (rare, but possible)

Recommended verified recovery test (runbook)

1 Record baseline on production (before test):

- table inventory (`public`)
- row counts for critical tables (`app_user` , `employer` , `caregiver` , etc.)
- checksum/fingerprint sample (optional)

2 Restore to new project from a specific timestamp/backup.

3 Run the same validation SQL on restored project and compare:

- `public` table list
- row counts
- key spot checks (latest `created_at`, known IDs)

4 Pass criteria:

- all expected `public` tables present 
- row counts within expected recovery-point window

Ask a follow up question...

I'll check recent platform and database logs for backup/restore-related errors on this restored project.

✓ Ran get_logs

✓ Ran get_logs

✓ Ran get_active_incidents

④ Reasoned

I'll run a quick schema inventory query to verify whether any user-defined objects exist outside `auth`.

<> SCHEMA TABLE INVENTORY

select n.nspname as schema_name, count(c.oid) filter (where c.relkind in ('r','p')) as ta

schema_name	table_count
auth	23
extensions	0
graphql	0
graphql_public	0
public	0
realtime	3

④ Reasoned

What I verified

I checked the restored project directly:

- `public` has 0 tables
- Only Supabase-managed schemas (`auth` , `storage` , `realtime` , etc.) have objects
- No active platform incidents were reported
- Branch-action logs were empty; Postgres log retrieval failed in this

I

The Assistant can make mistakes. Double check responses.

Ask a follow up question...

 I need help wiSubject: Restore to new project did not restore public schema I restored the latest scheduled physical backup to a new project. The restore completed and the new project is healthy, but the public schema contains no tables. The production project contains business tables such as app_user, employer, caregiver, and others. The restored project contains only Supabase system tables under auth. The query select tablename from pg_tables where schemaname = 'public'; returns no rows in the restored project. Please investigate why the restore did not include the public schema and data, and advise how to perform a verified recovery test.th my project

 Reasoned

I'll quickly gather official backup/restore behavior from Supabase docs, then validate what exists in your current project schemas.

✓ Ran load_knowledge

✓ Ran search_docs

✓ Ran list_tables

 Reasoned

I'll check recent platform and database logs for backup/restore-related errors on this restored project.

✓ Ran get_logs

✓ Ran get_logs

✓ Ran get_active_incidents

 Reasoned

I'll run a quick schema inventory query to verify whether any user-defined objects exist outside `auth`.

<> SCHEMA TABLE INVENTORY

```
select n.nspname as schema_name, count(c.oid) filter (where c.relkind in ('r','p')) as ta
```

Assistant wants to run this query

Skip Run Query

   

The Assistant can make mistakes. Double check responses.

Ask a follow up question...

gpt-5.3-codex 

